

Ex. No. 07	APPLICATION STATE MANAGEMENT AND SERIALIZATION
Date of Exercise	
Git Hub Host Link	https://github.com/Karthikeyan-0/flutter.git

Aim

To manage application state and serialize/deserialize data using JSON in a Flutter application.

Description

This experiment demonstrates application state management using `setState()` and JSON serialization/deserialization using a `Student` model. Student data is converted from a Dart object into JSON format using `toJson()` and converted back into a Dart object using `fromJson()`. A list of students is displayed dynamically using `ListView.builder()`.

Procedure:

1. A new Flutter project named `state_app` was created using the Flutter command.
2. A Dart file named `student.dart` was created inside the `lib` folder to define the `Student` model.
3. The `Student` class was created with the properties `id`, `name`, and `course`.
4. The `toJson()` method was implemented to convert the `Student` object into a JSON-compatible `Map`.
5. The `fromJson()` factory constructor was implemented to convert JSON data back into a `Student` object.
6. A `Student` object was created with the values 1, Ragu, and CSE.
7. The `toJson()` method was used to serialize the `Student` object into JSON data.
8. The `fromJson()` method was used to deserialize the JSON data back into a `Student` object.
9. A `List<String>` was created to maintain the student names as application state.
10. The `setState()` method was used to add a student to the list whenever the Add Student button was pressed.
11. `ListView.builder()` was used to dynamically display the student names.
12. The application was executed and tested in the Chrome browser.

Program**student.dart**

```
class Student {  
  
  final int id;  
  
  final String name;  
  
  final String course;  
  
  Student({  
  
    required this.id,  
  
    required this.name,  
  
    required this.course,  
  
  });  
  
  Map<String, dynamic> toJson() {  
  
    return {  
  
      'id': id,  
  
      'name': name,  
  
      'course': course,  
  
    };  
  
  }  
  
  factory Student.fromJson(Map<String, dynamic> json) {  
  
    return Student(  
  
      id: json['id'],  
  
      name: json['name'],  
  
      course: json['course'],  
  
    );  
  
  }  
}
```

```
);  
  
}  
  
}
```

main.dart

```
import 'package:flutter/material.dart';  
  
import 'student.dart';  
  
  
void main() {  
  
  runApp(const StateApp());  
  
}  
  
  
class StateApp extends StatelessWidget {  
  
  const StateApp({super.key});  
  
  
  @override  
  
  Widget build(BuildContext context) {  
  
    return MaterialApp(  
  
      debugShowCheckedModeBanner: false,  
  
      title: 'State Management',  
  
      home: const StudentPage(),  
  
    );  
  
  }  
  
}
```

```
class StudentPage extends StatefulWidget {  
  
  const StudentPage({super.key});  
  
  @override  
  
  State<StudentPage> createState() => _StudentPageState();  
}  
  
class _StudentPageState extends State<StudentPage> {  
  
  List<String> students = [];  
  
  Student student = Student(  
  
    id: 1,  
  
    name: 'Raj',  
  
    course: 'CSE',  
  
  );  
  
  @override  
  
  void initState() {  
  
    super.initState();  
  
    Map<String, dynamic> data = student.toJson();  
  
    print(data);  
  
    Student newStudent = Student.fromJson(data);  
  
    print(newStudent.name);  
  }  
  
  void addStudent() {  
  
    setState(() {  
  
      students.add('Raj');  
  
    });  
  }  
}
```

@override

Widget build(BuildContext context) {

 return Scaffold(

 appBar: AppBar(

 title: const Text('Student State'),

),

 body: Column(

 children: [

 const SizedBox(height: 20),

 ElevatedButton(

 onPressed: addStudent,

 child: const Text('Add Student'),

),

 Expanded(

 child: ListView.builder(

 itemCount: students.length,

 itemBuilder: (context, index) {

 return ListTile(

 title: Text(students[index]),

);

 },

),

),

],

),

Output

The screenshot displays the DartPad web interface. On the left, the Dart code for a Flutter application is shown, implementing a simple state management system for a student. The code defines a `MyApp` class that builds a `MaterialApp` with a title of 'State Management' and a primary color of blue. It also defines a `StudentPage` class that manages a list of students. The `initState` method creates a student object with ID 1, name 'KARTHIKEYAN', and course 'CSE'. The code is as follows:

```
38
39 class MyApp extends StatelessWidget {
40   const MyApp({super.key});
41
42   @override
43   Widget build(BuildContext context) {
44     return MaterialApp(
45       debugShowCheckedModeBanner: false,
46       title: 'State Management',
47       theme: ThemeData(
48         primarySwatch: Colors.blue,
49       ),
50       home: const StudentPage(),
51     );
52   }
53 }
54
55 class StudentPage extends StatefulWidget {
56   const StudentPage({super.key});
57
58   @override
59   State<StudentPage> createState() => _StudentPageState();
60 }
61
62 class _StudentPageState extends State<StudentPage> {
63   List<String> students = [];
64
65   @override
66   void initState() {
67     super.initState();
68
69     // Create Student object
70     Student student = Student(
71       id: 1,
72       name: 'KARTHIKEYAN',
73       course: 'CSE',
74     );
75
76     // Convert object to JSON
```

On the right, the application's output is shown. The title is 'Student State Management'. Below the title, there is a list with one item: '1 KARTHIKEYAN'. At the bottom of the output area, the following JSON data is displayed:

```
Student JSON: {id: 1, name: KARTHIKEYAN, course: CSE}
JSON String: {"id":1,"name":"KARTHIKEYAN","course":"CSE"}
Student Name: KARTHIKEYAN
```

The bottom of the image shows the Windows taskbar with the search bar, task view button, and several application icons. The system tray on the right indicates the temperature is 35°C, the language is ENG IN, and the date is 08-09-2026.

Result

Thus, the Flutter application was successfully developed to manage changing student data using `setState()`. The Student object was successfully serialized into JSON and deserialized back into a Dart object. The student data was also dynamically displayed using `ListView.builder()`.